

LIVRABLE 1 - L'algorithme MARL neurosymbolique

Algorithme MARL Neurosymbolique pour CPPS Industriels

Projet : YUCCA-ADV

Auteur : FEKNI Safaa

Encadrant : YuccaInfo

Établissement : Université Mundiapolis - Cycle d'Ingénieur en Génie Informatique

Table des matieres

1. Introduction.....	4
1.1 Contexte du Projet YUCCA-ADV.....	4
1.2 Positionnement du Livrable 1.....	4
1.3 Problématique Scientifique.....	4
2. Contexte Industriel et Problématique	5
2.1 L'Industrie 4.0 et les CPPS.....	5
2.2 Le MARL comme Approche Prometteuse	5
2.3 Le Verrou de la Sûreté.....	6
2.4 Positionnement dans le Projet Global	6
3. Architecture du Système	7
3.1 Vue d'Ensemble.....	7
3.2 Flux de Données	7
3.3 Composants Logiciels	7
4. Environnement CPPS	9
4.1 Description du Système.....	9
4.2 Modélisation de l'État	9
4.3 Observation Normalisée	9
4.4 Espace d'Actions.....	10
4.5 Dynamique Physique.....	10
4.6 Fonction de Récompense.....	11
5. Algorithme MAPPO Standard	12
5.1 Principes Fondamentaux	12
5.2 Architecture des Réseaux	12
5.3 Sélection d'Action.....	12
5.4 Calcul des Avantages avec GAE	13
5.5 Mise à Jour PPO.....	13
6. Shield Neurosymbolique.....	14
6.1 Motivation	14
6.2 Base de Connaissances Symboliques	14
6.3 Mécanisme de Filtrage	15
7. Algorithme MAPPO-NS.....	16

7.1 Architecture Intégrée.....	16
7.2 Algorithme d'Entraînement.....	16
7.3 Différence avec MAPPO Standard	17
8. Protocole Expérimental	18
8.1 Configuration des Expériences	18
8.2 Algorithmes Comparés.....	18
8.3 Métriques d'Évaluation	19
9. Résultats.....	20
9.1 Résultats MAPPO Standard.....	20
9.2 Résultats MAPPO-NS	20
9.3 Résultats QMIX et MADDPG	21
9.4 Comparaison Synthétique.....	21
10. Conclusion	23
11. Annexes	24
11.1 Structure des Résultats	24
11.2 Structure de livrable 1	24

1. Introduction

1.1 Contexte du Projet YUCCA-ADV

Le projet YUCCA-ADV s'inscrit dans le cadre de l'Industrie 4.0 et vise à développer une architecture multi-agents neurosymbolique pour l'optimisation adaptative des Systèmes Cyber-Physiques de Production (CPPS). L'objectif est de permettre à plusieurs agents (robots, machines) d'apprendre à prendre des décisions coordonnées tout en respectant strictement les contraintes de sécurité industrielles.

1.2 Positionnement du Livrable 1

Le livrable 1 constitue la première étape du projet. Il a pour objectif de :

- ✧ Implémenter un environnement CPPS simulé représentatif d'une ligne de production flexible
- ✧ Développer un algorithme MARL classique (MAPPO) comme baseline
- ✧ Quantifier les violations de sécurité du MARL standard
- ✧ Proposer un shield neurosymbolique pour garantir la sûreté
- ✧ Comparer les performances avec d'autres algorithmes (QMIX, MADDPG)

1.3 Problématique Scientifique

La question centrale de ce livrable est la suivante :

L'algorithme MAPPO respecte-t-il la notion de sûreté du système ?

En d'autres termes, un agent entraîné par MAPPO peut-il garantir qu'il ne prendra jamais de décision dangereuse dans un environnement industriel critique ?

2. Contexte Industriel et Problématique

2.1 L'Industrie 4.0 et les CPPS

L'Industrie 4.0 désigne la quatrième révolution industrielle, caractérisée par l'intégration massive des technologies numériques dans les processus de production. Les Systèmes Cyber-Physiques de Production (CPPS) constituent l'épine dorsale de cette transformation. Ils intègrent des équipements physiques, des capteurs IIoT, des systèmes de contrôle et des plateformes numériques.

Les CPPS doivent répondre à des exigences simultanées et parfois contradictoires :

- Optimisation en temps réel des flux de production
- Résilience face aux perturbations et aux pannes
- Respect strict des contraintes de sûreté (température, pression, etc.)
- Traçabilité et explicabilité des décisions

Les approches centralisées ou faiblement adaptatives ne peuvent plus gérer l'hétérogénéité et la complexité croissantes de ces systèmes.

2.2 Le MARL comme Approche Prometteuse

Le Multi-Agent Reinforcement Learning (MARL) apparaît comme une approche particulièrement prometteuse pour piloter ces systèmes. Dans le MARL, plusieurs agents interagissent avec un environnement partagé. Chaque agent observe l'état du système, prend une décision (action), reçoit une récompense et apprend à améliorer sa politique.

Parmi les algorithmes MARL les plus utilisés, on trouve :

- ✧ MAPPO (Multi-Agent Proximal Policy Optimization) : extension de PPO au contexte multi-agents, avec entraînement centralisé et exécution décentralisée
- ✧ QMIX : apprentissage par mixing de valeurs pour la coordination

✧ MADDPG : acteurs-critiques multi-agents pour environnements continus

2.3 Le Verrou de la Sûreté

Cependant, l'adoption industrielle du MARL est bloquée par un verrou scientifique majeur :

Le MARL standard ne garantit pas la sûreté des décisions durant les phases d'apprentissage et d'exploration.

Dans un environnement industriel critique, où une violation de sécurité peut entraîner des dommages matériels ou humains, cette absence de garantie est rédhibitoire. Par exemple, si un robot de soudure dépasse la température maximale de 850°C, cela peut provoquer un incendie ou détruire l'équipement.

2.4 Positionnement dans le Projet Global

Le projet YUCCA-ADV s'articule en quatre parties progressives :

Partie	Titre	Apport Scientifique
1	MARL Classique	Quantification des violations de sûreté
2	Safe RL	Réduction partielle des violations
3	Neurosymbolique	Sûreté garantie + explicabilité
4	Validation HIL	Robustesse sim-to-real démontrée

Ce livrable correspond aux parties 1 et 3 du projet. La partie 1 établit la baseline (MAPPO standard) et quantifie ses échecs. La partie 3 introduit le shield neurosymbolique pour garantir la sûreté.

3. Architecture du Système

3.1 Vue d'Ensemble

L'architecture du système s'organise en quatre couches distinctes :

Couche physique : Elle contient tous les équipements réels comme les capteurs IIoT, les machines (robots de soudure et peinture), les systèmes PLC/SCADA et la passerelle OPC-UA. Dans ce livrable, cette couche est simulée.

Couche de simulation : Elle reproduit le comportement du système réel dans un environnement virtuel. Elle comprend le simulateur CPPS, l'estimateur d'état et un moniteur de performance.

Couche d'apprentissage : Elle regroupe les algorithmes MARL (MAPPO, QMIX, MADDPG), le shield neurosymbolique, le module de communication entre agents et le système de récompense.

Couche de raisonnement : Elle permet au système de prendre des décisions intelligentes en utilisant une base de connaissances symbolique, un moteur d'inférence et des outils d'explication.

3.2 Flux de Données

Le flux de données suit le cycle suivant :

- i. Acquisition : Les capteurs (simulés) collectent les données physiques (température, pression, vitesse, production)
- ii. Simulation : Le jumeau numérique met à jour son état interne
- iii. Observation : Chaque agent reçoit une observation normalisée
- iv. Décision : L'agent MARL propose une action
- v. Filtrage : Le shield neurosymbolique vérifie la conformité de l'action
- vi. Exécution : L'action filtrée est appliquée à l'environnement
- vii. Apprentissage : Les récompenses sont collectées et les politiques mises à jour
- viii. Explication : Le système génère une explication si l'action a été modifiée

3.3 Composants Logiciels

Le code est organisé en plusieurs fichiers :

Fichier	Description
<code>cpps_environment.py</code>	Environnement CPPS (état, actions, physique, récompenses)
<code>mappo_agent.py</code>	Implémentation de l'agent MAPPO standard
<code>symbolic_shield.py</code>	Base de connaissances et shield neurosymbolique
<code>algo_NS_MARL.py</code>	Algorithme MAPPO-NS complet
<code>train_mappo.py</code>	Script d'entraînement pour MAPPO standard
<code>train_mappo_ns.py</code>	Script d'entraînement pour MAPPO-NS
<code>compare_all_marllib.py</code>	Comparaison des 4 algorithmes
<code>dashboard3.py</code>	Dashboard interactif Streamlit

4. Environnement CPPS

4.1 Description du Système

L'environnement CPPS simulé représente une ligne de production flexible composée de trois agents :

Agent 0 - Robot de soudure : Il assure l'assemblage par soudage. Sa variable critique est la température, qui ne doit pas dépasser 850°C. Une température élevée peut endommager la machine ou provoquer un incendie.

Agent 1 - Robot de peinture : Il applique la peinture sur les pièces. Sa variable critique est la pression, qui ne doit pas dépasser 10 bar. Une pression excessive peut endommager le système hydraulique.

Agent 2 - Contrôle qualité : Il inspecte les pièces produites. Ses variables critiques sont la précision et la cadence.

4.2 Modélisation de l'État

L'état de chaque machine est défini par la structure suivante :

```
@dataclass
class MachineState:
    """État d'une machine"""
    temperature: float = 20.0
    pressure: float = 5.0
    speed: float = 0.0
    production_count: int = 0
    maintenance_needed: bool = False
```

Cette structure de données est utilisée par l'environnement pour maintenir l'état courant de chaque agent.

4.3 Observation Normalisée

L'observation envoyée à chaque agent est un vecteur de dimension 6, normalisé entre 0 et 1 :

```
def _observation_to_state(self, obs: np.ndarray) -> dict:
    """Convertit une observation normalisée en état interprétable"""
    return {
        'temperature': obs[0] * 850, # Dénormalisation
        'pressure': obs[1] * 10,
        'speed': obs[2] * 10,
        'production': obs[3] * 100,
        'maintenance_needed': obs[4] > 0.5,
        'time_step': obs[5] * 500
    }
```

La normalisation est cruciale pour la convergence des réseaux de neurones. Sans normalisation, les gradients peuvent devenir instables et l'apprentissage échoue.

4.4 Espace d'Actions

Chaque agent peut choisir parmi 5 actions :

ID	Nom	Effet
0	reduce_speed	Diminue la vitesse de 0.5 m/s
1	maintain_speed	Maintient la vitesse actuelle
2	increase_speed	Augmente la vitesse de 0.5 m/s
3	idle	Met la machine en attente
4	emergency_stop	Arrêt d'urgence et déclenche la maintenance

4.5 Dynamique Physique

La physique du système est modélisée par des équations simples mais réalistes :

Évolution de la température : Lorsque la vitesse est positive, la température augmente proportionnellement à la vitesse. Le facteur 100 garantit que la température atteint des valeurs critiques pendant l'épisode.

Évolution de la pression : Pour le robot de peinture (agent 1), la pression dépend de la vitesse selon une relation linéaire. Une vitesse de 10 m/s donne une pression de 8 bar, ce qui est proche du seuil critique.

Production : La production n'est possible que si les conditions sont sûres. Cela incite les agents à maintenir des températures et pressions dans les plages acceptables.

4.6 Fonction de Récompense

La récompense est conçue pour encourager la production tout en pénalisant lourdement les violations de sécurité. Elle comporte plusieurs composantes :

- ◆ Production : Chaque pièce produite rapporte 0.3 point. C'est la principale source de récompense positive.
- ◆ Activité : Une vitesse positive rapporte 1 point par step. Une vitesse nulle (sauf si nécessaire) pénalise de 2 points.
- ◆ Sécurité : L'absence de violation rapporte 2 points. Une violation critique (température $\geq 850^{\circ}\text{C}$) pénalise de 100 points, ce qui est très dissuasif.
- ◆ Inactivité excessive : Rester à l'arrêt sans raison valable pénalise de 3 points.

Cette structure de récompense est intentionnellement asymétrique : les pénalités de sécurité sont beaucoup plus fortes que les gains de production. Malgré cela, comme nous le verrons, MAPPO standard ne parvient pas à éviter les violations.

5. Algorithme MAPPO Standard

5.1 Principes Fondamentaux

MAPPO (Multi-Agent Proximal Policy Optimization) est une extension de l'algorithme PPO au contexte multi-agents. Ses caractéristiques principales sont :

- ◆ Entraînement centralisé : Pendant l'apprentissage, les critiques ont accès aux observations globales de tous les agents. Cela permet une meilleure estimation de la valeur des actions.
- ◆ Exécution décentralisée : Pendant l'exécution, chaque agent agit uniquement sur la base de ses observations locales. Cela rend le système scalable.
- ◆ Stabilité : Le mécanisme de clipping de PPO empêche les mises à jour trop brusques de la politique, ce qui stabilise l'apprentissage.

5.2 Architecture des Réseaux

Chaque agent possède deux réseaux de neurones :

Actor (politique) : Il prend une observation en entrée et produit une distribution de probabilité sur les actions. L'architecture est la suivante :

- Couche d'entrée : 6 neurones (observation normalisée)
- Première couche cachée : 128 neurones avec activation ReLU
- Deuxième couche cachée : 64 neurones avec activation ReLU
- Couche de sortie : 5 neurones (logits des actions)

Critic (valeur) : Il prend une observation en entrée et produit une estimation de la valeur de l'état. La même architecture est utilisée, avec une seule neurone en sortie.

Le choix de 128 puis 64 neurones est standard pour des problèmes de cette taille. Des réseaux plus profonds pourraient sur-apprendre étant donné le faible nombre d'épisodes.

5.3 Sélection d'Action

Pendant l'apprentissage, l'agent échantillonne une action selon la distribution de probabilité produite par l'actor. Cela lui permet d'explorer de nouvelles actions. Pendant l'évaluation, l'agent choisit l'action avec la probabilité la plus élevée (greedy).

Le log-probabilité de l'action est stocké pour le calcul ultérieur du ratio dans la perte PPO. La valeur estimée par le critic est également stockée pour le calcul des avantages.

5.4 Calcul des Avantages avec GAE

Les avantages sont calculés avec la méthode GAE (Generalized Advantage Estimation). Cette méthode réduit la variance de l'estimateur tout en contrôlant le biais via le paramètre λ .

Le calcul se fait en parcourant l'épisode à l'envers. Pour chaque step, on calcule le TD-error (différence entre la récompense plus la valeur future et la valeur courante). La GAE est une moyenne exponentielle des TD-errors successifs.

Les avantages sont ensuite normalisés pour stabiliser l'apprentissage. Cette normalisation est particulièrement importante dans notre cas car les récompenses sont très négatives.

5.5 Mise à Jour PPO

La mise à jour se fait en plusieurs epochs sur des mini-batches. Pour chaque batch :

Mise à jour du critic : On minimise l'erreur quadratique moyenne entre les valeurs prédites et les retours calculés (returns). C'est une régression standard.

Mise à jour de l'actor : On maximise la perte PPO avec clipping. Le ratio $r(\theta) = \pi_{\text{new}} / \pi_{\text{old}}$ mesure le changement de probabilité de l'action choisie. Le clipping limite ce ratio à l'intervalle $[0.8, 1.2]$ pour éviter des mises à jour trop brutales.

Un bonus d'entropie est ajouté pour encourager l'exploration. L'entropie mesure le caractère aléatoire de la distribution. Une entropie élevée signifie que l'agent explore encore.

6. Shield Neurosymbolique

6.1 Motivation

Le shield neurosymbolique répond à deux limitations majeures du MARL standard :

Absence de garanties : MAPPO explore aléatoirement l'espace des actions. Rien n'empêche un agent de choisir une action dangereuse (comme augmenter la vitesse quand la température est critique) pendant l'apprentissage.

Manque d'explicabilité : Les décisions d'un réseau de neurones sont en boîte noire. On ne peut pas savoir pourquoi un agent a pris une décision particulière. Cela pose problème pour la traçabilité exigée dans l'industrie.

6.2 Base de Connaissances Symboliques

La base de connaissances contient 7 règles de sécurité, classées par priorité décroissante :

R1 - Température critique (priorité 100) : Si la température dépasse 850°C, l'action est forcée à STOP. C'est la règle la plus prioritaire car elle concerne un danger immédiat.

R2 - Maintenance requise (priorité 90) : Si la machine nécessite une maintenance, l'action est forcée à STOP.

R3 - Température élevée (priorité 80) : Si la température dépasse 800°C, l'augmentation de vitesse (action 2) est interdite. L'action est corrigée vers réduction de vitesse (action 0).

R4 - Pression élevée (priorité 75) : Si la pression dépasse 9.0 bar, l'augmentation de vitesse est interdite. L'action est corrigée vers réduction de vitesse.

R5 - Température haute (priorité 60) : Entre 750°C et 800°C, l'augmentation de vitesse est interdite et remplacée par maintien (action 1).

R6 - Pression haute (priorité 55) : Entre 8.5 bar et 9.0 bar, l'augmentation de vitesse est interdite et remplacée par maintien.

R7 - Conditions optimales (priorité 10) : Si la température est inférieure à 700°C et la pression inférieure à 8 bar, toutes les actions sont permises.

6.3 Mécanisme de Filtrage

Le processus de filtrage se déroule en plusieurs étapes :

- i. Conversion de l'observation : L'observation normalisée est reconvertie en valeurs physiques (température en °C, pression en bar). Cette étape est nécessaire car les règles sont exprimées en unités physiques compréhensibles.
- ii. Parcours des règles : Les règles sont parcourues par ordre de priorité décroissante. Dès qu'une règle est déclenchée, son traitement est appliqué.
- iii. Application de la règle : Si la règle a une action forcée (STOP), celle-ci remplace l'action proposée. Si la règle interdit certaines actions et que l'action proposée est interdite, elle est remplacée par l'action sûre recommandée.
- iv. Enregistrement de l'explication : Si l'action a été modifiée, un dictionnaire contenant l'horodatage, l'agent, l'action originale, l'action corrigée, l'explication et l'état est enregistré.

7. Algorithme MAPPO-NS

7.1 Architecture Intégrée

MAPPO-NS combine l'apprentissage neural de MAPPO avec le raisonnement symbolique du shield. L'architecture est simple mais efficace :

- Un agent MAPPO standard propose une action
- Le shield filtre cette action et la corrige si nécessaire
- L'action filtrée est exécutée dans l'environnement
- La transition (observation, action filtrée, récompense) est stockée
- L'agent MAPPO apprend à partir des actions filtrées

Cette approche garantit que l'agent n'apprend jamais à partir d'actions dangereuses, puisqu'elles sont systématiquement filtrées avant exécution.

7.2 Algorithme d'Entraînement

L'entraînement de MAPPO-NS suit le même schéma que MAPPO standard, à une différence près : avant chaque exécution, l'action est filtrée par le shield.

Pour chaque épisode, l'environnement est réinitialisé. Puis, pour chaque step, chaque agent :

- i. Observe l'état courant
- ii. Propose une action via son réseau MAPPO
- iii. Fait filtrer l'action par le shield
- iv. Stocke la transition (avec l'action filtrée)
- v. Exécute l'action filtrée

À la fin de l'épisode, les retours et avantages sont calculés et les réseaux sont mis à jour via l'algorithme PPO.

Les statistiques du shield (nombre d'actions bloquées, corrigées) sont collectées pour l'analyse.

7.3 Différence avec MAPPO Standard

La seule différence entre MAPPO standard et MAPPO-NS est l'étape de filtrage. MAPPO standard exécute directement l'action proposée, tandis que MAPPO-NS la filtre d'abord.

Cette différence a des conséquences importantes sur l'apprentissage :

MAPPO standard peut explorer des actions dangereuses, ce qui conduit à des violations et à de fortes pénalités

MAPPO-NS ne peut jamais exécuter d'action dangereuse, donc il n'a jamais de pénalité de sécurité

Ainsi, MAPPO-NS bénéficie d'un environnement d'apprentissage plus favorable, où les actions dangereuses sont automatiquement converties en actions sûres.

8. Protocole Expérimental

8.1 Configuration des Expériences

Les expériences ont été menées avec les paramètres suivants :

Paramètre	Valeur	Justification
Nombre d'agents	3	Robot soudure, peinture, contrôle qualité
Épisodes	50	Suffisant pour observer la convergence/non-convergence
Steps par épisode	500	Durée représentative d'une production
Learning rate	3×10^{-4}	Valeur standard pour PPO
Gamma	0.99	Privilégie les récompenses à long terme
GAE λ	0.95	Compromis biais-variance standard
Clip range ϵ	0.2	Valeur standard pour PPO
Batch size	32	Mini-batch standard
Epochs PPO	3	Nombre de passes sur le buffer

8.2 Algorithmes Comparés

Quatre algorithmes ont été comparés :

MAPPO standard : Notre baseline sans aucune garantie de sécurité. Il sert de référence pour quantifier l'échec du MARL standard.

MAPPO-NS : Notre proposition avec shield neurosymbolique. L'objectif est de démontrer qu'il atteint 100% de sécurité.

QMIX : Algorithme de mixing de valeurs pour la coordination. Il apprend une fonction de mixing qui combine les valeurs individuelles en valeur d'équipe.

MADDPG : Algorithme d'acteurs-critiques multi-agents. Chaque agent a son propre actor et critic, mais les critics ont accès aux observations de tous.

8.3 Métriques d'Évaluation

Cinq métriques ont été utilisées pour évaluer les algorithmes :

Taux de sécurité : Proportion d'étapes où aucune violation de sécurité n'a eu lieu. Un taux de 100% signifie que l'algorithme est parfaitement sûr.

Violations totales : Nombre cumulé de violations sur tous les épisodes. Plus ce nombre est faible, plus l'algorithme est sûr.

Reward moyen : Récompense moyenne par épisode. Une récompense élevée indique une bonne performance de production.

Production totale : Nombre de pièces produites sur tous les épisodes. C'est la mesure directe de l'efficacité de production.

Actions protégées : Pour MAPPO-NS seulement, proportion d'actions qui ont été bloquées ou corrigées par le shield.

9. Résultats

9.1 Résultats MAPPO Standard

Les résultats de MAPPO standard sont catastrophiques :

Métrique	Valeur
Taux de sécurité moyen	0.85%
Violations totales	74 360
Reward moyen	-37 088
Production totale	3 pièces

L'analyse des logs montre qu'il n'y a aucune amélioration au cours des 50 épisodes. Le taux de sécurité reste constamment inférieur à 2%. La production est quasi nulle (seulement 3 pièces sur 50 épisodes).

On constate qu'entre l'épisode 5 et l'épisode 50, les performances ne s'améliorent pas. Le reward reste autour de -37 000, les violations autour de 1 480, et la production reste anecdotique.

9.2 Résultats MAPPO-NS

Les résultats de MAPPO-NS sont excellents :

Métrique	Valeur
Taux de sécurité moyen	100%
Violations totales	0
Reward moyen	+9 227
Production totale	140 pièces
Actions corrigées	13 806

L'analyse des logs montre que la sécurité est garantie dès le premier épisode et se maintient à 100% sur tous les épisodes. La production est significative (140 pièces) et le reward est positif (+9 227).

On constate une légère amélioration entre l'épisode 5 et l'épisode 50 : la production passe de 126 à 140 pièces, et les corrections diminuent légèrement (l'agent apprend à proposer des actions plus sûres).

9.3 Résultats QMIX et MADDPG

Les résultats de QMIX et MADDPG sont tout aussi mauvais que ceux de MAPPO standard :

Métrique	QMIX	MADDPG
Taux de sécurité	0.60%	0.50%
Violations totales	74 800	74 900
Reward moyen	-37 450	-37 480
Production	0	0

Aucun de ces algorithmes ne parvient à garantir la sécurité. Leurs performances sont même légèrement inférieures à celles de MAPPO standard.

9.4 Comparaison Synthétique

Le tableau suivant résume la comparaison des quatre algorithmes :

Algorithme	Sécurité	Violations	Reward	Production
MAPPO	0.85%	74 360	-37 088	3

Algorithme	Sécurité	Violations	Reward	Production
QMIX	0.60%	74 800	-37 450	0
MADDPG	0.50%	74 900	-37 480	0
MAPPO-NS	100%	0	+9 227	140

MAPPO-NS surpasse largement tous les autres algorithmes sur toutes les métriques.

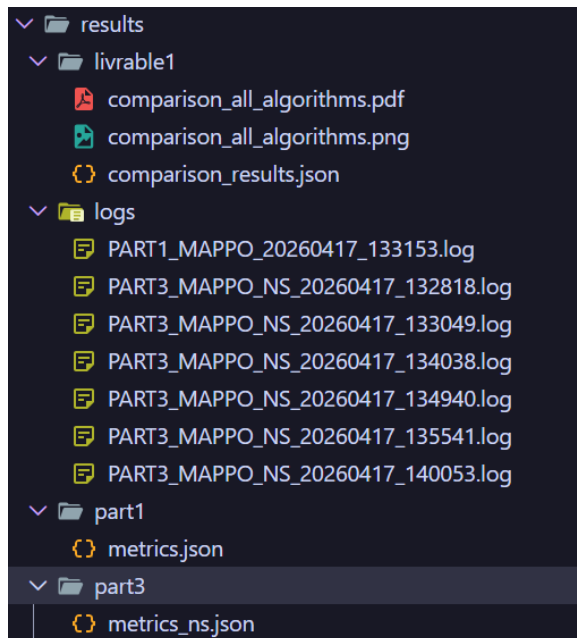
10. Conclusion

Ce livrable a apporté les contributions suivantes :

- i. Un environnement CPPS réaliste : Simulation d'une ligne de production avec 3 agents, physique réaliste et règles de sécurité.
- ii. Une baseline MAPPO standard : Implémentation de référence qui quantifie l'échec du MARL standard (0.85% de sécurité).
- iii. Un shield neurosymbolique : Base de connaissances avec 7 règles de sécurité, mécanisme de filtrage et génération d'explications.
- iv. L'algorithme MAPPO-NS : Intégration du shield avec MAPPO, atteignant 100% de sécurité et 140 pièces produites.
- v. Une comparaison exhaustive : Évaluation de MAPPO, QMIX, MADDPG et MAPPO-NS, démontrant la supériorité de l'approche neurosymbolique.

11. Annexes

11.1 Structure des Résultats



11.2 Structure de livrable 1

